

**Insufficient Cryptography, Poor Code Quality,  
Code Tampering & Reverse Engineering**

# Introduction

In modern mobile security, the integrity and confidentiality of applications are constantly threatened by systemic weaknesses rooted in development and deployment practices. This report explores four critical areas that expose applications to risk: **Insufficient Cryptography**, which occurs when mechanisms are improperly implemented or used, leading to the exposure of sensitive data; **Poor Code Quality**, which introduces vulnerabilities that invite security risks and exploitation; and the associated adversarial techniques, **Reverse Engineering**, which analyzes a software system to identify its core components, and **Code Tampering**, which involves unauthorized modification of binary resources.

The combination of these flaws creates significant entry points for attackers, emphasizing the need for a comprehensive, multi-layered security approach to prevent unauthorized access, data breaches, and system compromise.

## What is Insufficient Cryptography?

Insufficient cryptography occurs when cryptographic mechanisms are improperly implemented or used in mobile applications, leading to the exposure of sensitive data.

## What is Poor Code Quality?

Poor code quality refers to vulnerabilities within the application code that invite security risks and exploitation.

## What is Code Tampering?

Code tampering involves unauthorized modification of binary resources.

## What is Reverse Engineering?

Reverse engineering is the process of analyzing a software system to identify its core components.

## **Vulnerabilities and Research Methods**

- **Insufficient Cryptography Methods**
  - Vulnerabilities often stem from a lack of encryption on sensitive data, the use of weak cryptography in hardcoded strings, and the creation of custom encryption protocols. Further risks include the use of insecure or deprecated algorithms like MD5 or SHA1.
- **Poor Code Quality and Reverse Engineering**
  - Poor code quality manifests through a lack of input validation, hardcoded credentials, and inadequate code obfuscation. Reverse engineering methods involve using tools like Apktool to decompile APKs and identify hidden secrets.

## **Impact and Consequences**

- **Data Compromise and Exploitation**
  - Insufficient cryptography makes sensitive information easily accessible to attackers, leading to data breaches and unauthorized transactions.
- **Unauthorized Access and Binary Tampering**
  - Poor code quality allows for unauthorized users to gain access to restricted areas and facilitates the alteration of data being transmitted. Tampering can bypass license checks or inject malicious scripts.

# **Case Study**

## **Weak Cryptography and Hardcoded Secrets in a Messaging App**

A secure messaging application claimed end-to-end encryption but was found to be using a hardcoded AES key within its source code to encrypt local message databases.

### **The Vulnerability (Insufficient Cryptography & Poor Code Quality)**

The application utilized a static, hardcoded cryptographic key for sensitive data storage. Additionally, the app failed to implement code obfuscation, making it trivial for researchers to locate the key during analysis.

### **The Attack (Reverse Engineering)**

A researcher performed reverse engineering using JADX to decompile the APK. By searching for cryptographic classes, they identified the hardcoded AES key and the specific algorithm used, which was an outdated version of SHA1 for hashing.

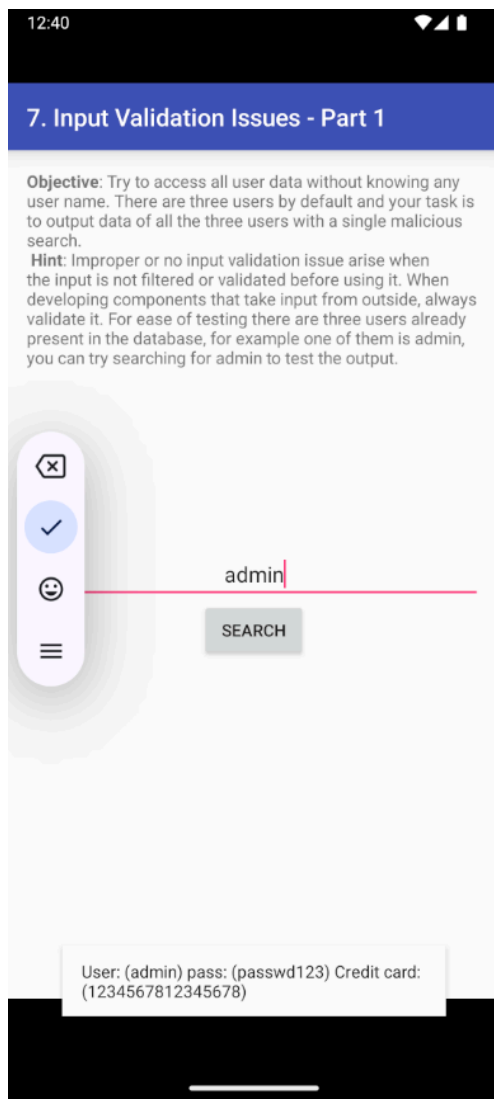
### **Result**

The discovery meant that any attacker with physical or remote access to the device's storage could decrypt every message stored by the user, highlighting how poor implementation nullifies even the strongest theoretical encryption.

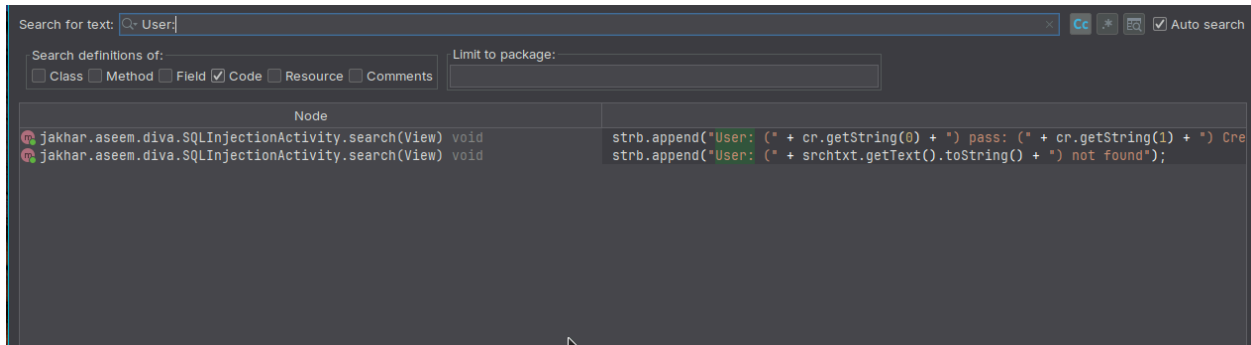
# Practical (DIVA Lab Lab APK)

## Input validation PART 1

As mentioned in the hint, we first try entering **admin** which shows us the admin password and credit card number



We decompile the app using **Jadx** and search for the string “User: ” which shows us:



```
public void search(View view) {
    EditText srchtxt = (EditText) findViewById(R.id.ivilsearch);
    try {
        Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText()
        StringBuilder strb = new StringBuilder("");
        if (cr != null && cr.getCount() > 0) {
            cr.moveToFirst();
            do {
                strb.append("User: (" + cr.getString(0) + ") pass: (" + cr.getString(1) + ") Cred
            } while (cr.moveToNext());
        } else {
            strb.append("User: (" + srchtxt.getText().toString() + ") not found");
        }
        Toast.makeText(this, strb.toString(), 0).show();
    } catch (Exception e) {
        Log.d("Diva-sqli", "Error occurred while searching in database: " + e.getMessage());
    }
}
```

We see that it is not validating the input while constructing the sql statement. Meaning, we can perform SQL Injection.

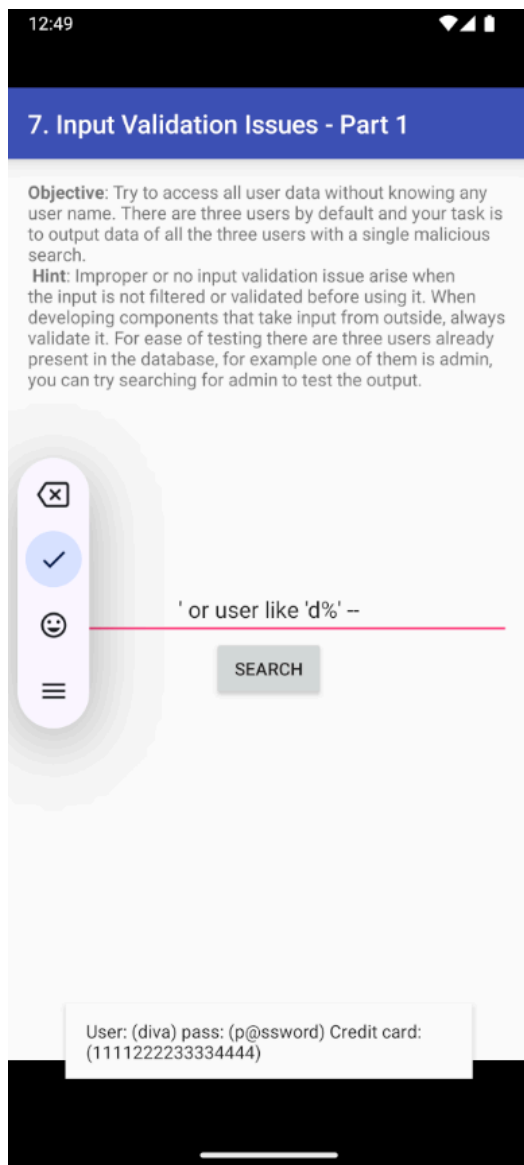
```
Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText().
```

We enter:

' or user like 'a%' --

Which basically means ignore all other conditions and show users that start with the character "a".

We repeat this for each letter in the alphabet until we see several users listed with their password and credit card number.



12:49



## 7. Input Validation Issues - Part 1

**Objective:** Try to access all user data without knowing any user name. There are three users by default and your task is to output data of all the three users with a single malicious search.

**Hint:** Improper or no input validation issue arise when the input is not filtered or validated before using it. When developing components that take input from outside, always validate it. For ease of testing there are three users already present in the database, for example one of them is admin, you can try searching for admin to test the output.

The image shows a mobile application interface. On the left, there is a vertical sidebar with four icons: a back arrow, a checkmark, a smiley face, and a hamburger menu. The main area contains a search bar with the placeholder text "' or user like 'j%' -". Below the search bar is a grey button labeled "SEARCH".

User: (john) pass: (password123) Credit card:  
(5555666677778888)

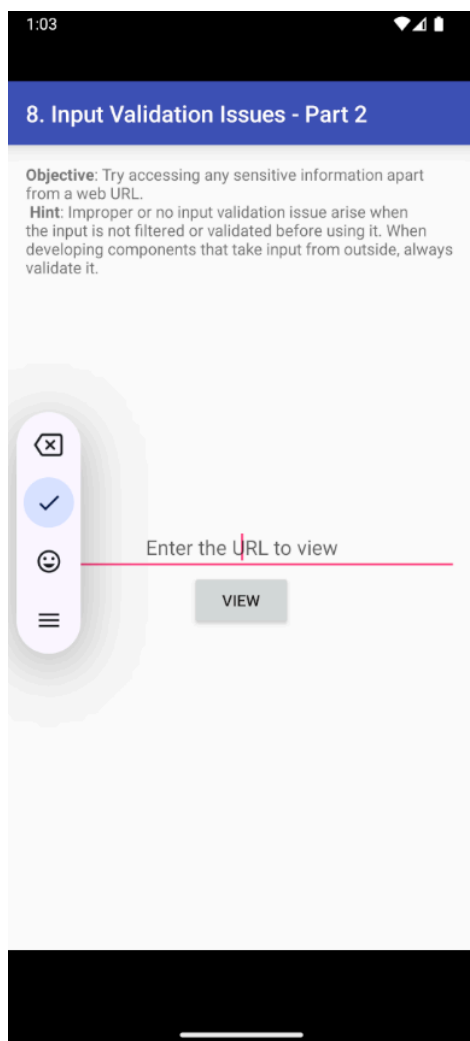


# Practical (DIVA Lab Lab APK)

## Input validation PART 2

Goal: Open a url that isn't a http request.

Since the input is not validated properly we can try opening the app's internal files present in its data dir which is otherwise inaccessible.



First we use adb with root to see what is present in the app's data directory

```
> ~/Downloads via = via ● via 🐼 via 🦉 v3.13.12 via ⚡ default via t
Z) adb root
adb is already running as root

> ~/Downloads via = via ● via 🐼 via 🦉 v3.13.12 via ⚡ default via t
Z) adb shell
emu64xa:/ # ls /data
```

```
jakhar.aseem.diva/
emu64xa:/ # ls /data/data/jakhar.aseem.diva/
app_textures/  app_webview/  cache/        code_cache/    databases/     shared_prefs/
emu64xa:/ # ls /data/data/jakhar.aseem.diva/shared_prefs/
AwOriginVisitLoggerPrefs.xml  WebViewChromiumPrefs.xml
emu64xa:/ # ls /data/data/jakhar.aseem.diva/shared_prefs/
```

We see that it has a file WebViewChromiumPrefs.xml in shared\_prefs where secrets are typically stored.

It is found in path:

**/data/data/jakhar.aseem.diva/shared\_prefs/WebViewChromiumPrefs.xml**

There is no validation for url protocol so we can use “file://” to access local files.

By entering the following url:

**file:///data/data/jakhar.aseem.diva/shared\_prefs/WebViewChromiumPrefs.xml**

The app will happily print the file's content.

file:///data/data/jakhar.aseem.diva/shared\_pre

VIEW

This XML file does not appear to have any style information associated with it. The document tree is shown below.

---

```
▼ <map>
  <int name="lastVersionCodeUsed"
    value="636771938"/>
</map>
```

# Mitigations

- **Standard Cryptographic Libraries**
  - Avoid custom encryption protocols; use industry-standard, well-vetted libraries and strong random number generation.
- **Code Obfuscation and Hardening**
  - Implement robust code obfuscation to hinder reverse engineering and utilize platform-specific secure storage for credentials.
- **Input Validation and Secure Transport**
  - Enforce strict input validation on the server side and utilize modern TLS protocols for all data in transit.

## Conclusion

The combination of insufficient cryptography and poor code quality creates significant entry points for attackers. When applications lack proper identity verification, permission checks, or robust data encryption, they become vulnerable to large-scale data theft. Furthermore, the susceptibility to code tampering and reverse engineering emphasizes the need for a multi-layered security approach that includes secure coding practices, anti-tampering mechanisms, and the avoidance of hardcoded secrets.

# References

<https://owasp.org/www-project-mobile-top-10/2023-risks/m10-insufficient-cryptography>

<https://owasp.org/www-project-mobile-top-10/2016-risks/m7-client-code-quality>

<https://owasp.org/www-project-mobile-top-10/2016-risks/m8-code-tampering>

<https://owasp.org/www-project-mobile-top-10/2016-risks/m9-reverse-engineering>